

摘要：

MiniMax在启动科创板IPO辅导后，推出新一代旗舰模型M3，首次同时实现顶尖编程能力、1M超长上下文与原生多模态。M3可独立复现ICLR获奖论文，12小时内跑通核心实验。通过全新稀疏注意力机制与交互式训练框架，其Agent与代码优化能力显著提升。

MiniMax启动科创板IPO辅导后，新一代旗舰模型M3也正式登场。

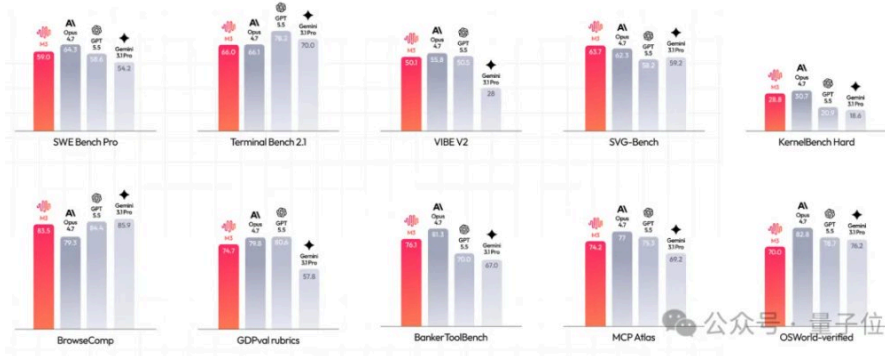
刚刚发布的M3，把前沿Coding能力、1M超长上下文、原生多模态这三条科技树，同时点满。

MiniMax称，M3是目前的开源模型当中，唯一能同时做到这三点的。



它在SWE-Bench Pro上超过了GPT-5.5和Gemini 3.1 Pro，接近Claude Opus 4.7。

而在面向自主Agent的端到端评测Claw-Eval上，M3拿到了最高分。



为了证明三种能力确实同时在线，MiniMax团队把一篇ICLR 2025获奖论文丢给M3独立复现

- 。论文又长又硬、图文并茂，要看懂里面的曲线图、数据、公式得靠多模态；
- 。论文、代码、实验日志一次性塞进窗口得靠长上下文；
- 。长线程甚至并发地跑完复现得靠编程和agent能力。

结果M3独立跑了接近12小时，最终成功把论文的核心实验全部跑通。

现在，MiniMax M3已经可以在MiniMax Code、Token Plan和API服务中体验到。

长上下文、编程、多模态同时在线

和M系列以往的路径相同，M3同时点满长上下文、Coding和多模态这三条科技树，依然是在向Coding和Agent方向重点发力。

MiniMax用几道真题测了它的实际表现。

FP8矩阵乘（GEMM）优化是大模型推理里优化难度最高的环节之一，资深工程团队通常要集中投入1-2周才能在NVIDIA Hopper架构上写出一个生产级kernel。

MiniMax把这道题丢给M3，起点只有一份任务描述、一个benchmark脚本、一个跑不起来的Triton骨架，没有任何参考实现可以抄。

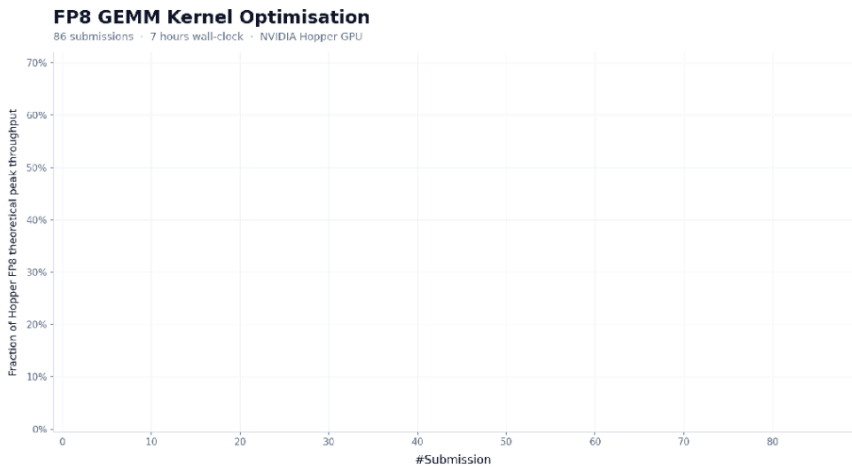
随后约24小时，M3自己走完了从baseline到生产级优化的全部路径，其间完成了147次benchmark提交、1959次工具调用。

并且，性能瓶颈诊断、CUDA Graph集成、persistent kernel重写等一系列步骤中，M3在每一步都拿benchmark反馈来验证自己。

最终经过6轮标志性优化，Hopper FP8硬件峰值利用率从首版7.6%干到了71.3%，实现9.4倍加速。

执行过程本身也值得关注。

其余模型大多在前30次提交内就不再进展、主动退出了，M3的最优解出现在第145次提交，在那之前它经历了多个性能平台期，但一直在继续尝试不同方向。



除了优化算子，M3还能独立复现论文。

MiniMax把一篇ICLR 2025 Outstanding Paper Award获奖论文扔给它，这篇论文研究的是大模型微调过程中的学习动力学。

LEARNING DYNAMICS OF LLM FINETUNING

Yi Ren
University of British Columbia
renyi.joshua@gmail.com

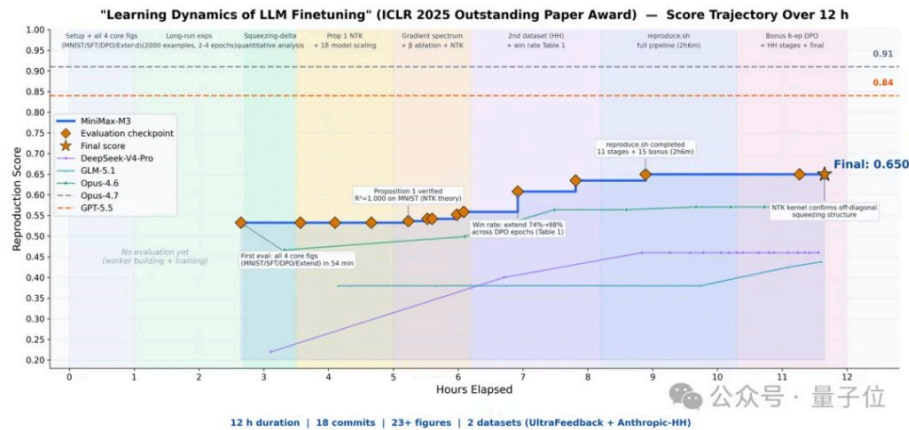
Danica J. Sutherland
University of British Columbia & Amii
dsuth@cs.ubc.ca

论文本身又长又硬、图文并茂，模型需要看懂里面的曲线图、数据、公式然后编写代码并执行。

这个任务重，M3自主运行接近12小时，产出18次commit与23张实验图表。

它不仅跑通了核心实验，成功吻合了SFT阶段的预测概率变化趋势，还清晰观测到DPO实验重点讨论的squeezing效应，并顺利验证了原论文提出的Extend缓解方法。

中途遇到跑不通的实验，它会自己进行诊断，碰到结果对不上的地方就自己调整，整个过程始终没有人工介入。



除此之外，M3还能给其他模型当教练。

MiniMax在PostTrainBench上让它接手四个只完成预训练的Base模型，任务是在12小时内自主完成数据合成、训练、评测、迭代的全部流程，最终让这些模型在数学推理、工具调用、科学知识推理、代码生成等任务上具备基本能力。

这道题没有清晰的反馈结构，也没有标准答案，M3需要自己判断合成什么数据、选什么训练策略，并根据每轮评测结果，决定下一步怎么调。

它先分析每个Base模型在各任务上的短板，再决定为每个方向合成哪些数据，跑完训练拿到评测结果之后，再自己判断哪个任务还差得远、哪个方向的数据质量不够、下一轮应该往哪里加力。

如果发现某个任务的数据覆盖不足，它会重新设计合成方案；某轮训练效果不达预期，它会调整策略再跑一轮。

不同轮次的评测结果出来之后，它会自己对比差异、总结规律、决定下一步。

整个12小时它都在自己拿主意，没有人告诉它该怎么做。

它最终得分0.37，略低于Opus 4.7（0.42）和GPT-5.5（0.39），但明显领先其余模型。



引入全新注意力机制

要解决更复杂的Agent任务，context scaling是绕不开的挑战。

MiniMax为此从最底层的注意力机制入手，自己做了一套新的稀疏注意力架构，叫**MSA**（MiniMax Sparse Attention）。

全注意力机制有个先天问题，它的计算复杂度随上下文长度平方级增长。上下文越长，计算量爆炸得越快。

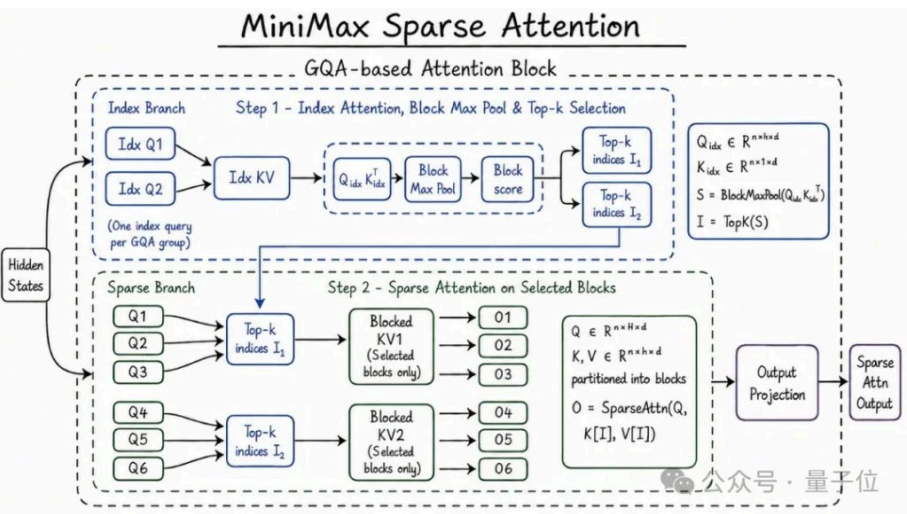
业内通常靠增加初筛阶段来缓解这个问题，但精度和效率很难兼顾。

MSA的做法分两步。

- 第一步是**Index Attention**，用轻量的索引query对KV块做Block Max Pool，选出Top-k的高相关块；
- 第二步，再对选出来的块做完整的**Sparse Attention**计算。

跟DSA、MoBA等方案相比，MSA可以更精确地为KV分块，实现更高的有效上下文覆盖。

算子层面，MiniMax也做了专门优化，采用以KV块为外层聚合命中query的KV outer gather Q方式，每块只读一次、访存连续。



在M3的head配比下，这种模式的计算访存比显著优于通行方法，比开源的Flash-Sparse-Attention、FlashMoBA快4倍以上。

效果上，在1M上下文下，**M3每token的计算量只有上代模型的1/20**，prefilling阶段提速超过9倍，decoding阶段提速超过15倍，且绝大部分能力与全注意力持平。

多模态方面，M3**从预训练第一步就做图文混合训练**，不同模态数据的语义空间从一开始就在同一套框架下融合。

MiniMax做了大量实验，发现文本和图像在序列中交替排列的interleaved data，对模型性能的提升比通常认为的更关键。

为了大规模获取这类数据，MiniMax重构了整套数据管线，训练数据Token规模因此可以提升到100万亿量级。

Coding能力这边，MiniMax认为今天模型的Coding水平，越来越取决于能否用真实世界的用户逻辑来训练。

大多数代码Agent的训练和评测建立在单轮任务的假设上，但真实场景里用户往往在同一个session里持续协作。

为了缩小这个差距，MiniMax构建了一套交互式用户模拟器框架，模拟真实开发者在协作过程中的行为模式，让模型在训练阶段就接触到更接近生产环境的交互场景。

至于更多技术细节，MiniMax预告其技术报告和开源模型权重将在10天内更新，感兴趣的话可以持续关注。

本文来源：[量子位](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限免

 30  收藏

分享到:  

写评论

请发表您的评论

 表情  图片

发表评论

